Textora Bulk SMS & API Gateway

Architectural & Technical Specifications

---

1. System Overview

Textora is a high-speed, enterprise-grade Bulk SMS broadcast platform and developer API gateway. It operates as a multi-channel SMS management system:

1. **Consumer Portal (Telegram & WhatsApp Bots)**: A conversational interface that enables individuals and business owners to upload contact lists, compose messages, customize sender identities, and execute bulk SMS campaigns on a pay-as-you-go basis. Textora is fully integrated and accessible on both **Telegram** and **WhatsApp**.
2. **Developer API (REST Gateway)**: A lightweight, developer-focused API that allows external systems to integrate SMS capabilities (such as One-Time Passwords, alerts, and customer notifications) directly into their custom backend codebases using a prepaid balance model.

Textora includes built-in concurrency protection, local persistent transaction locking, and a self-healing background worker that recovers from server crashes without dropping messages or double-billing clients.

---

2. Custom Sender ID Policies (Zero LOA Requirement)

To register custom Sender IDs (alphanumeric brand names, up to 11 characters, used as the sender name for SMS broadcasts), the following guidelines apply:

- **No Letter of Authorization (LOA) Required**: Unlike traditional SMS gateways that require clients to upload corporate registry documents, power of attorney, or signed Letters of Authorization (LOA) to register a brand name, **Textora does not require an LOA in any case**.
- **Seamless Approval**: Users request their custom Sender IDs directly through the Telegram or WhatsApp bot. The administration processes and approves these requests directly on the downstream telecommunication gateway without requesting legal documentation.
- **Pricing**: A flat, one-time setup fee (configured via CUSTOM_SENDER_ID_FEE_GHS) is paid via Paystack. Once approved, the Sender ID is unlocked permanently for that user's account.

---

3. Phone Number Validation & Tiered International Pricing

Textora supports sending SMS messages to both local Ghana numbers and international destinations:

3.1. Formatting & Normalization

- **Ghana Numbers (+233)**: Inputted local numbers (10 digits starting with 0 or 9 digits) are sanitized, cleaned of non-numeric characters, and prefixed with the country code 233.
- **International Numbers**: Any cleaned phone number with a length between 7 and 15 digits is classified as a valid international number. The platform preserves the inputted country prefix (e.g., 1... for US, 44... for UK, 234... for Nigeria).

3.2. Tiered Pricing Model

SMS delivery costs are calculated dynamically based on the classification of the recipient:

1. **Local SMS (Ghana destinations)**:
    - **Bot campaigns**: Charged at the configured retail rate (SELLING_PRICE_PER_SMS, e.g., GHS 0.06).
    - **Developer API**: Deducts **1.0 unit** per message segment (equivalent to GHS 0.05).
2. **International SMS (Non-Ghana destinations)**:
    - **Bot campaigns**: Charged at a flat rate of **GHS 0.08** per message segment.
    - **Developer API**: Deducts **1.6 units** per message segment (since the international unit rate is GHS 0.08, and the base developer prepaid unit value is GHS 0.05, $0.08 / 0.05 = 1.6$ credits).

---

4. API Endpoints (REST Specification)

Developers authenticate by providing their API Key in the HTTP headers:

- Authorization: Bearer <API_KEY> or x-api-key: <API_KEY>.

4.1. Get Balance

- **URL**: GET /api/v1/balance
- **Description**: Returns the remaining prepaid SMS units balance.
- **Response**:

```
{
  "ok": true,
  "balance": 1450.00
}
```

4.2. List Verified Sender IDs

- **URL**: GET /api/v1/sender-ids
- **Description**: Returns the list of sender names approved for this account.
- **Response**:

```
{
  "ok": true,
  "senderIds": ["MYBRAND", "ALERT"]
}
```

4.3. Send SMS

- **URL**: POST /api/v1/send-sms
- **Description**: Sends messages to one or more recipients.
- **Request Body**:

```
{
  "to": ["233201234567", "1234567890"],
  "message": "Verify your login with code: 8876.",
  "senderId": "MYBRAND"
}
```

- **Response Details**:
    - The local recipient (233201234567) is charged at **1.0 credit** per segment.
    - The international recipient (1234567890) is charged at **1.6 credits** per segment.
- **Response JSON**:


```
{
  "ok": true,
  "total_units_debited": 2.6,
  "remaining_balance": 1447.40,
  "results": [
    { "recipient": "233201234567", "status": "success", "data": { "code": "ok", "message": "Submitted successfully" } },
    { "recipient": "1234567890", "status": "success", "data": { "code": "ok", "message": "Submitted successfully" } }
  ]
}
```


4.4. Fetch API Logs

- **URL**: GET /api/v1/logs
- **Description**: Returns the developer's API SMS transmission logs.
- **Response**:


```
{
  "ok": true,
  "logs": [
    {
      "userId": "12345678",
      "timestamp": "2026-08-14T09:45:00.000Z",
      "message": "Verify your login with code: 8876.",
      "recipientsCount": 2,
      "unitsDebited": 2.6,
      "remainingBalance": 1447.40,
      "status": "success"
    }
  ]
}
```


4.5. Request Recharge

- **URL**: POST /api/v1/recharge
- **Description**: Creates a Paystack payment session to purchase SMS credits.
- **Request Body**:

```
{
  "planId": 3
}
```

- **Response**:

```
{
  "ok": true,
  "plan_id": 3,
  "amount_ghs": "100.00",
  "credits": 2000,
  "checkout_url": "https://checkout.paystack.com/...",
  "reference": "ref_abc123"
}
```

---

5. Database Schemas (Data Storage Specification)

5.1. api-developers.json

Stores registered developers, mapping their API keys and Telegram/WhatsApp user IDs:

```
{
  "api_keys": {
    "txt_abcdef1234567890": {
      "userId": "12345678",
      "apiKey": "txt_abcdef1234567890",
      "balance": 1447.40,
      "name": "Kwadwo Code"
    }
  },
  "users": {
    "12345678": {
      "userId": "12345678",
      "apiKey": "txt_abcdef1234567890",
      "balance": 1447.40,
      "name": "Kwadwo Code"
    }
  }
}
```

5.2. api-usage-log.json

Maintains records of API transmissions:

```
[
 {
  "userId": "12345678",
  "timestamp": "2026-08-14T09:45:00.000Z",
  "message": "Verify your login with code: 8876.",
  "recipientsCount": 2,
  "unitsDebited": 2.6,
  "remainingBalance": 1447.40,
  "status": "success"
 }
]
```


---

6. Concurrency Protection & Recovery

6.1. Local File Transaction Locking (lockManager.js)

To avoid race conditions when multiple API calls or webhooks hit the server at the same millisecond, Textora uses a locking mechanism:

- Before reading/writing any shared JSON store (e.g. api-developers.json or payment-sessions.json), the thread must acquire a unique file lock (locks/<lockName>.lock).
- If a lock file exists, other threads wait, checking at a regular interval (retryIntervalMs = 50ms) until it is released or expires (lockExpiryMs = 60s).

6.2. Crash Recovery Worker (recoveryWorker.js)

When bulk SMS campaigns are broadcasted, they might consist of tens of thousands of contacts. If the server crashes mid-broadcast:

- The system writes individual progress states into sms-log.json for each contact.
- When the server boots up, it starts the **Textora Recovery Worker**.
- The worker scans payment-sessions.json for any transactions marked as processing that have been active for more than 120 seconds.
- It iterates over the campaign contacts, checks sms-log.json to see which recipients have already received the message, and **continues the broadcast for the remaining contacts** without duplicating any messages.